

CS2300: File Structures and Introduction to Database Systems

Functional Dependency

San Yeung

Why Do We Normalize?



Database Design Methods

GOAL: Information preservation (for attributes, entities, and relations) and minimum redundancy.

Bottom-up Design Methodology

Start with individual attributes and then consider the relations between them.

Top-down Design Methodology

Start with grouping of attributes by natural relation and then decompose the relations.

Two Goals



Information Preservation

the concepts that are originally captured
in the conceptual design after the
conceptual to logical design mapping



Minimum Redundancy

minimizing redundant storage of the
same information
reducing the need for multiple updates
to maintain consistency across multiple
copies of the same information

Quiz

Which schema design contains more “goodness”, i.e. one set of groupings of attributes is better than another?

A: (student_id, course, student_name, grade)

B: (student_id, course, grade), (student_id, student_name)

A) A

B) B

C) They are both good

D) Others

Relational DB Schema Design Optimization

1. How to determine whether or not a schema is ***“well-designed”***
2. **EER-Relational Schema Mapping** is CRITICAL!
3. Data Preservation + Integrity + Anomaly-free Operations
4. Minimum Data Redundancy and NULL values



Design Guidelines

1. Clear Semantics of attributes and schema
2. Reduced Redundancy
3. Reduced NULL values
4. Disallow possibility of generating Spurious tuples

Clear Semantics of attributes and schema

- Make the schema relation easy to explain.
- Do not combine attributes from multiple entities.
- **Example:** Let's assume all the data for **Band**, **Album**, **Song**, and **Release_Year** exist only in this *single* table in the database.

Band_Songs			
Band	Album	Song	Release Year
Kirby Krackle	E Is For Everyone	Great Lake Adventures	2010
Kirby Krackle	E Is For Everyone	Ring Capacity	2010
Kirby Krackle	Kirby Krackle	Back to the Beginning	2009
Kirby Krackle	Super Powered Love	I Wanna Live In A World of Super Heroes	2011
Kirby Krackle	Super Powered Love	In Another Castle	2011

Reduced Redundancy

- Design the base relation schema to avoid insertion, deletion and modification anomalies.
- Insertion anomalies
 - Insertion needs duplicated values or NULL values.
- Deletion anomalies
 - Deletion can cause loss of information from the database.
- Modification anomalies
 - Changing one attribute value can lead to other tuples being incorrect

Insertion Anomalies

Ex: EMP_DEPT(ENAME, SSN, BDATE,
ADDRESS, DNUMBER,
DNAME, DMGRSSN)

Redundancy!

...	SSN	...	DNUMBER	DNAME	DMGRSSN
	123		1	HR	333
	???		2	Math	123

Input errors could be very common

Deletion Anomalies

Ex: EMP_DEPT(ENAME, SSN, BDATE,
ADDRESS, DNUMBER,
DNAME, DMGRSSN)

EMP_DEPT					Redundancy	
Ename	Ssn	Bdate	Address	Dnumber	Dname	Dmgr_ssn
Smith, John B.	123456789	1965-01-09	731 Fondren, Houston, TX	5	Research	333445555
Wong, Franklin T.	333445555	1955-12-08	638 Voss, Houston, TX	5	Research	333445555
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle, Spring, TX	4	Administration	987654321

If you delete the last employee working for a particular dept., then all info about that dept. is lost!

Modification Anomalies

Ex: EMP_DEPT(ENAME, SSN, BDATE,
ADDRESS, DNUMBER,
DNAME, DMGRSSN)

...	SSN	...	DNUMBER	DNAME	DMGRSSN
	123		1	HR	333
	345		1	Service	333

If you want to modify **DNAME for a particular dept., then have to change several tuples!**

Informal Measures of Quality

- *Design the base relation schemas so that no insertion, deletion, or modification anomalies are present*
 - *If any anomalies are present, make sure that the programs that update the database will operate correctly*
 - *These guidelines may sometimes have to be violated in order to improve some query performances (e.g. materialized view for constant query)*
- *Bottom line: use anomaly-free base relations when possible*

Reduced NULL values

- Avoid NULL values
- When you do use them, make sure they are an exception and not the norm.

Musician_Songs

Musician	Band	Album	Song	Release Year
Jonathan Coulton	NULL	Thing a Week Three	Code Monkey	2006
Jonathan Coulton	NULL	Many Hands: Family Music for Haiti	Princess Who Saved Herself	2010
Zearle	NULL	Hackers and Crackers	Class War	NULL
NULL	Kirby Krackle	Kirby Krackle	Back to the Beginning	2009
NULL	Kirby Krackle	Super Powered Love	I Wanna Live In A World of Super Heroes	2011

Disallow possibility of generating Spurious tuples

- Design relation schemas so they can be joined with equi-joins on primary keys or foreign keys
- Joins that result in “wrong” or “invalid” info – **prevent!**

• **Ex:**

<u>STUDENT</u>	<u>INSTRUCTOR</u>	<u>STUDENT</u>	<u>COURSE</u>
Smith	Lin	Smith	CS 238
Smith	Wilkerson	Smith	CS 330

Join on STUDENT can produce:
(Smith, Lin, CS 330)

Disallow possibility of generating Spurious tuples

- When tables are designed (normalized) correctly, every join operation should not introduce any unintended or extra rows that lack valid basis.

- Joins that result in “wrong” or “invalid” info – **prevent!**

- Ex:

<u>STUDENT</u>	<u>INSTRUCTOR</u>
Smith	Lin
Smith	Wilkerson

<u>STUDENT</u>	<u>COURSE</u>
Smith	CS 238
Smith	CS 330

Join on STUDENT can produce:
(Smith, Lin, CS 330)

Need ways to **formally** evaluate DB design as
“good” or “bad”

Functional Dependency

- A tool for analysis

Determining whether a schema is “GOOD”

Formal definition of a constraint (FD)

Deriving implicit FDs

Normal Forms (1NF, 2NF, 3NF, BCNF)

Fixing a “BAD” schema

Decompositions and their properties

Finding 3NF Decompositions

Finding BCNF Decompositions

Functional Dependency Analysis

A very useful tool to *formally* evaluate and prove that a RDB design is good or bad.

Definition

A **Functional Dependency** in a relation is a constraint-relationship that exists when one set of attributes **uniquely determines** another set of attributes.

In other words:

- If you know the **determinating value** of the first set, you can **predict** (or guarantee) the value of the other set.

This formal analysis is crucial for understanding and describing the **behavior/dependency** of data in a relational model.

Functional Dependencies

For any relation R, attribute B is functionally dependent on attribute A if, for every valid instance of A, that value of A uniquely determines the value of B.

A -> B
Left Hand Side Right Hand Side
Determinant

CONSULTANT_ID	NAME	CONTACT_INFO	SALARY
001	Alice	alice@gmail.com	90000
002	Bob	bob@gmail.com	75000
003	Alice	aliceSmith@gmail.com	85000
004	David	david@gmail.com	92000

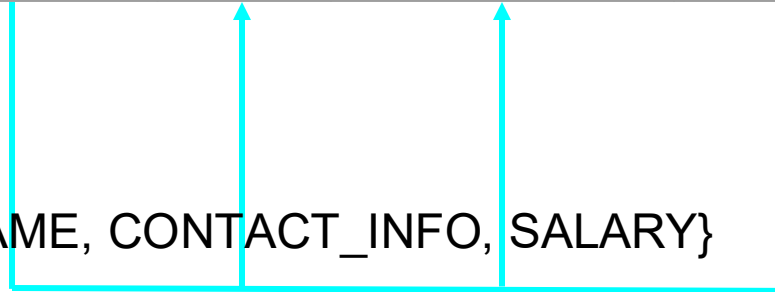
Functional Dependencies

For any relation R, attribute B is functionally dependent on attribute A if, for every valid instance of A, that value of A uniquely determines the value of B.

A -> B
Left Hand Side Right Hand Side
Determinant

CONSULTANT_ID	NAME	CONTACT_INFO	SALARY
---------------	------	--------------	--------

CONSULTANT_ID → {NAME, CONTACT_INFO, SALARY}



Functional Dependencies (FDs)

- **Constraint** between **2 sets** of attributes
- Let X, Y be non-empty sets of attributes in a relation R , FD $X \rightarrow Y$ says:
For any tuples t_1, t_2 in relation R ,
if $t_1[X] = t_2[X]$, then $t_1[Y] = t_2[Y]$.
- X is called left-hand side (**LHS**)
- Y is called right-hand side (**RHS**)
- single most important concept in **relational schema design theory**

Example

TUPLE #	A	B	C
1	10	b1	c1
2	10	b2	c2
3	11	b4	c1
4	12	b3	c4
5	13	b1	c1
6	14	b3	c4

Does $A \rightarrow B$?

No.

$t_1[A] = t_2[A]$, but
 $t_1[B] \neq t_2[B]$

Example

TUPLE #	A	B	C
1	10	b1	c1
2	10	b2	c2
3	11	b4	c1
4	12	b3	c4
5	13	b1	c1
6	14	b3	c4

Does $B \rightarrow C$?

Yes!

Look at tuples t_1 and t_5 , and tuples t_4 and t_6

Example

TUPLE #	A	B	C
1	10	b1	c1
2	10	b2	c2
3	11	b4	c1
4	12	b3	c4
5	13	b1	c1
6	14	b3	c4

Does $C \rightarrow B$?

No.

**$t_1[C] = t_3[C]$, but
 $t_1[B] \neq t_3[B]$**

Note

- If X is a **candidate** key of R
 - This implies that $X \rightarrow Y$ for any subset of attributes Y of R
 - So if X is a candidate key of R , then $X \rightarrow R$
- If $X \rightarrow Y$ holds in R , does this say whether or not $Y \rightarrow X$ is also in R ?

Examples

Relation: (SSN, Name, Address, Working_Dept)

SSN	Name	Address	Working_Dept
123-45-6789	John	123 Maple St	HR
234-56-7890	Maria	456 Oak Rd	Marketing
345-67-8901	Ahmad	789 Pine Ave	IT

Obvious FDs:

1. SSN $\rightarrow$ Name // SSN is the *determinant*, and Name is the *dependent*
2. SSN $\rightarrow$ Address
3. SSN $\rightarrow$ Working_Dept
4. SSN $\rightarrow$ {Name, Address, Working_Dept} : *consolidated form*

Examples

Relation: (SSN, Name, Address, Working_Dept)

SSN	Name	Address	Working_Dept
123-45-6789	John	123 Maple St	HR
234-56-7890	Maria	456 Oak Rd	Marketing
345-67-8901	Ahmad	789 Pine Ave	IT

Incorrect FDs:

1. **Name** → **SSN** (*Employees might share the same name*)
2. **Address** → **Working_Dept** (*obviously there are no dependency between them*)

Functional Dependencies

FDs are essential for the integrity and efficiency of the database during the design optimization/normalization process.

How are FDs determined?

<u>OrderID</u>	OrderDate	CustomerID	Customer Name	Customer Address	<u>ProductID</u>	Product Description	Product Finish	Unit Price	Ordered Quantity
----------------	-----------	------------	---------------	------------------	------------------	---------------------	----------------	------------	------------------

Functional Dependencies

FDs are determined from the inherent meaning and relationships within the data and based on the semantics and requirements of the database application.

Usually, a long-lasting collaborative effort is needed among the database designers, domain experts, business analysts, end users, etc.

<u>OrderID</u>	OrderDate	CustomerID	Customer Name	Customer Address	<u>ProductID</u>	Product Description	Product Finish	Unit Price	Ordered Quantity
----------------	-----------	------------	---------------	------------------	------------------	---------------------	----------------	------------	------------------

FDs must hold for all legal states
of a relation, not just current state!

So define FDs carefully!

FDs: property of the relation schema R

A functional dependency is not a property of a particular relation **state** r of R

- cannot be inferred automatically from a given relation extension
 - however, can use a single counterexample to disprove a functional dependency
- must be explicitly defined by someone who knows the semantics of the attributes of R
- Some FDs may not be obvious, but can be deduced from other FDs
- Given a set of FDs F for a relation R , the set of all FDs for R is **F^+** (known as the **closure** of F)

Inference Rules

(1) Reflexive rule: If Y is a subset of X ,
then $X \rightarrow Y$ (also $X \rightarrow X$)

Ex: (SSN, PNUMBER, HOURS, ENAME,
PNAME, PLOCATION)

**So {Hours, ENAME} $\rightarrow$ Hours,
PNAME $\rightarrow$ PNAME, etc.**

Trivial FDs

Inference Rules

(2) Augmentation rule:

If $X \rightarrow Y$, then $XZ \rightarrow YZ$

Ex: (SSN, PNUMBER, HOURS, ENAME,
PNAME, PLOCATION)

PNUMBER $\rightarrow$ PLOCATION

So {PNUMBER, SSN} $\rightarrow$ {PLOCATION, SSN}

Inference Rules

(3) Transitive rule: If $X \rightarrow Y$ and $Y \rightarrow Z$,
then $X \rightarrow Z$

Ex: (SSN, PNUMBER, HOURS, ENAME,
PNAME, PLOCATION, PBUILDINGSIZE)

**PNUMBER $\rightarrow$ PLOCATION,
PLOCATION $\rightarrow$ PBUILDINGSIZE**

Then PNUMBER $\rightarrow$ PBUILDINGSIZE

Armstrong's Inference Rules

- There are three other inference rules that follow from IR1, IR2, and IR3.
 - IR4 (decomposition rule):
 - $\{X \rightarrow YZ\} \models X \rightarrow Y$
 - IR5 (union rule):
 - $\{X \rightarrow Y, X \rightarrow Z\} \models X \rightarrow YZ$
 - IR6 (pseudotransitive rules):
 - $\{X \rightarrow Y, WY \rightarrow Z\} \models WX \rightarrow Z$

Armstrong's Inference Rules

- [Armstrong, 1974] showed that F^+ can be computed by repeatedly applying the 3 rules (**reflexive, augmentation, and transitive**) until no new dependencies added
- Computationally hard for large relations! Sometimes a small set of FDs are enough.
- A systematic way:
 - Find all X's that appear as a LHS of current FDs
 - Find the set of all attributes dependent on X
 - Closure of X

Closure of X under F (X^+)

- X^+ = set of all attributes dependent on X

Algorithm

1. start with $X^+ = X$
2. for each FD $Y \rightarrow Z$ in F do
 - if Y is a subset of X^+
 - then $X^+ = X^+ \cup Z$
3. Continue this process until no more attributes can be added to X^+

Example

Given a relation Student and a set of functional dependencies F as follows, compute the closure for all LHS.

Student(SID, dept, dept_chair)

$F = \{ \text{SID} \rightarrow \{\text{dept}, \text{dept_chair}\}, \text{dept} \rightarrow \text{dept_chair}, \{\text{SID}, \text{dept}\} \rightarrow \text{dept_chair} \}$

$\{\text{SID}\}^+ = \{\text{SID}, \text{dept}, \text{dept_chair}\}$

$\{\text{dept}\}^+ = \{\text{dept}, \text{dept_chair}\}$

$\{\text{SID}, \text{dept}\}^+ = \{\text{SID}, \text{dept}, \text{dept_chair}\}$

If the closure of a LHS include all attributes, this LHS is a **super key** of the relation.

Candidate Keys

- If X^+ = the set of all attributes in a relation R, and if there **does not exist** Y in X such that $(X - \{Y\})^+ = \text{all attributes in R}$, then X is a **candidate key** for R
- In previous example, $\{\text{SID}, \text{dept}\}^+$ includes all attributes, SID^+ also includes all attributes.
- **SID** is a candidate key

Equivalent Sets of FDs

- Two sets of FDs F_1 and F_2 are ***equivalent***, if every FD in F_1 can be inferred from F_2 and vice versa.
- A simple approach to check the equivalency
 - Compute the closures of all LHS in F_1 by using FDs in F_2 to see if F_2 can infer F_1 .
 - Similarly, we can check if F_1 infers F_2 .

Example of Equivalent FD sets

$F = \{ A \rightarrow C, AC \rightarrow D, E \rightarrow AD, E \rightarrow H \}$

$G = \{ A \rightarrow CD, E \rightarrow AH \}$

Can all FDs in F1 be inferred from using F2?

Example of Equivalent FD sets

$F = \{ A \rightarrow C, AC \rightarrow D, E \rightarrow AD, E \rightarrow H \}$

$G = \{ A \rightarrow CD, E \rightarrow AH \}$

Can all FDs in F2 be inferred from using F1?

Minimal Set (Cover) of FDs

F is ***minimal*** if it satisfies all these conditions:

- 1) Every FD in F has a single attribute for its RHS
(e.g., not $SID \rightarrow \{\text{dept}, \text{dept_chair}\}$)
- 2) Can't replace $X \rightarrow A$ in F with $Y \rightarrow A$ where Y is a **subset** of X, and still have equivalent set of FDs
(e.g., not $\{SID, \text{dept}\} \rightarrow \text{dept_chair}$)
- 3) Can't remove any FD from F and still have equivalent set of FDs (e.g., no FD in F that can be inferred from another FD in F)

Algorithm

Given a set of FDs F , find its minimal cover

- Step1: Decompose each FD to get single attribute at **RHS**
- Step2: For each FD, remove redundant attribute from **LHS**
- Step3: Remove redundant **FDs**
- **Optional Recombination**

Strict Algorithm

Algorithm 16.2. Finding a Minimal Cover F for a Set of Functional Dependencies E

Input: A set of functional dependencies E .

1. Set $F := E$.
2. Replace each functional dependency $X \rightarrow \{A_1, A_2, \dots, A_n\}$ in F by the n functional dependencies $X \rightarrow A_1, X \rightarrow A_2, \dots, X \rightarrow A_n$.
3. For each functional dependency $X \rightarrow A$ in F
for each attribute B that is an element of X
if $\{ \{F - \{X \rightarrow A\} \} \cup \{ (X - \{B\}) \rightarrow A \} \}$ is equivalent to F
then replace $X \rightarrow A$ with $(X - \{B\}) \rightarrow A$ in F .
4. For each remaining functional dependency $X \rightarrow A$ in F
if $\{F - \{X \rightarrow A\}\}$ is equivalent to F ,
then remove $X \rightarrow A$ from F .

Example

Given $F = \{B \rightarrow AB, D \rightarrow A, AB \rightarrow D\}$

- Step 1: $B \rightarrow AB$ is decomposed into $B \rightarrow A, B \rightarrow B$
($B \rightarrow B$ is trivial and is removed)
- Step 2: check if $AB \rightarrow D$ has redundant LHS. Can it be $A \rightarrow D$ or $B \rightarrow D$?

Compute AB^+ and B^+ based on **F**

$AB^+ = ABD$ and $B^+ = ABD$, so A is extraneous.

- So far, we have $F = \{B \rightarrow A, D \rightarrow A, B \rightarrow D\}$
 - Any transitive dependencies?

Example (cont.)

- So far, we have $F = \{B \rightarrow A, D \rightarrow A, B \rightarrow D\}$
- Step 3: check if there is any redundant FDs.

Is $B \rightarrow A$ redundant?

compute B^+ based on $F - \{B \rightarrow A\}$

$B^+ = BDA$, that means we can obtain $B \rightarrow A$ from

$F - \{B \rightarrow A\}$, so $B \rightarrow A$ is redundant.

Similarly, check the remaining FDs.

Final answer $F' = \{D \rightarrow A, B \rightarrow D\}$

Minimal Set (Cover) of FDs

- Can there be more than one minimal cover for a relation?
- Yes! And they **won't necessarily even have the same number of FDs** in them!
- Ex: (A, B, C)
 $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow A, AB \rightarrow C, AB \rightarrow BC\}$
 minimal set: $G_1 = \{A \rightarrow B, B \rightarrow C, C \rightarrow A\}$
 minimal set: $G_2 = \{A \rightarrow B, B \rightarrow A, B \rightarrow C, C \rightarrow B\}$

Full Functional Dependencies

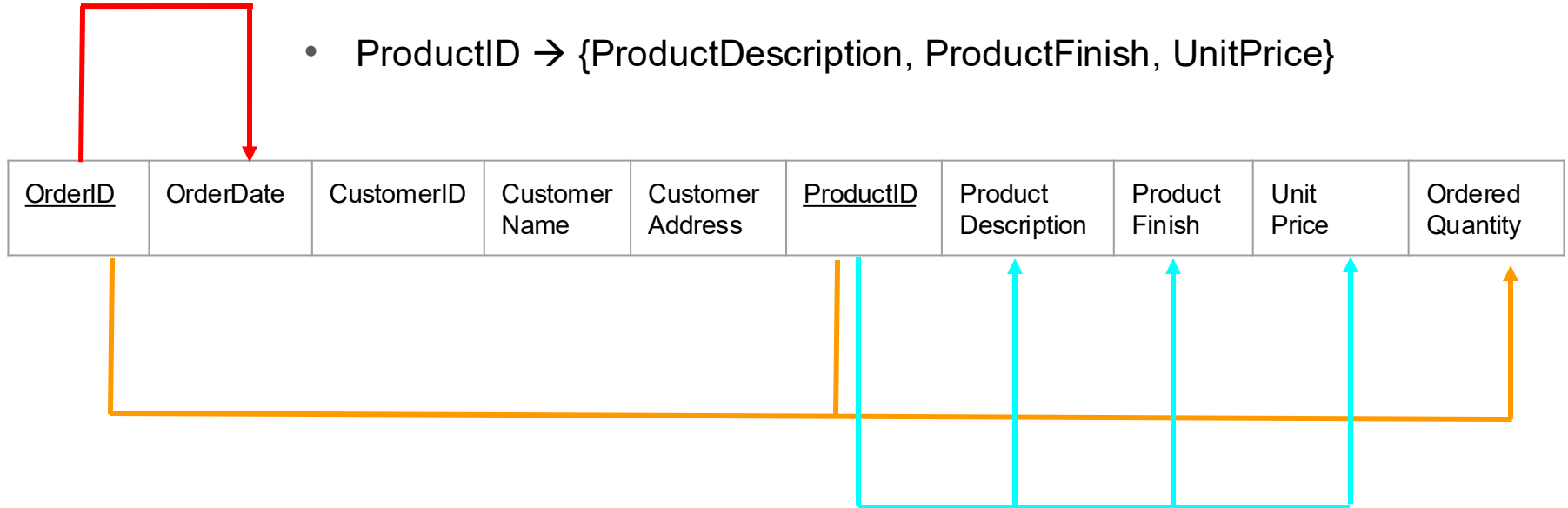
A full functional dependency occurs when a set of attributes (Y) is functionally dependent on the entire set of another attribute (X), and not on any proper subset of that set.

Formal Definition: A functional dependency $X \twoheadrightarrow Y$ is a **full functional dependency** if removal of any attribute A from X means that the dependency does not hold anymore; that is, for any attribute $A \in X$, $(X - \{A\})$ does *not* functionally determine Y.

X is the “**minimal set**” of attributes that Y depends on.

Full Functional Dependencies

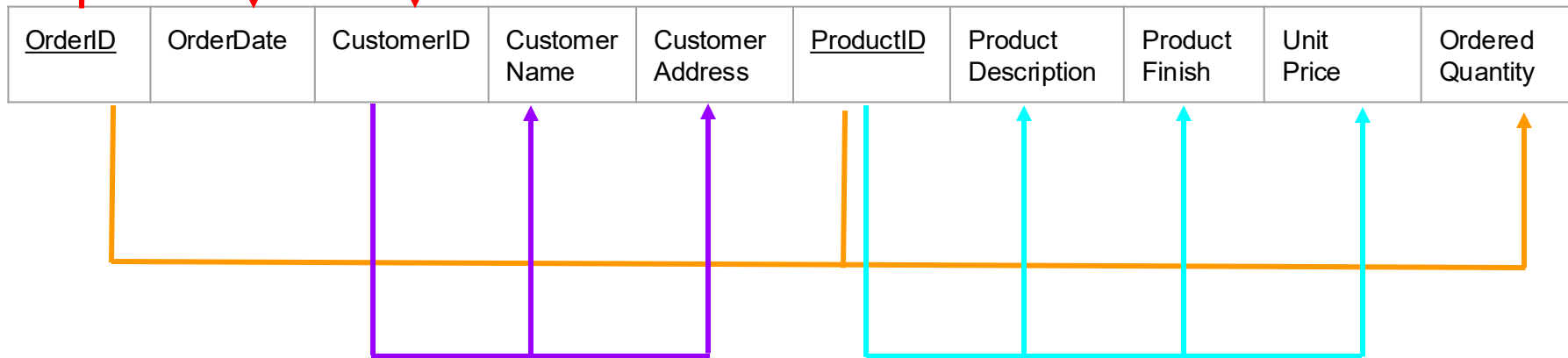
- $\text{OrderID} \rightarrow \text{OrderDate}$
- $\{\text{OrderID}, \text{ProductID}\} \rightarrow \text{OrderedQuantity}$
- $\text{ProductID} \rightarrow \{\text{ProductDescription}, \text{ProductFinish}, \text{UnitPrice}\}$



Partial Functional Dependency

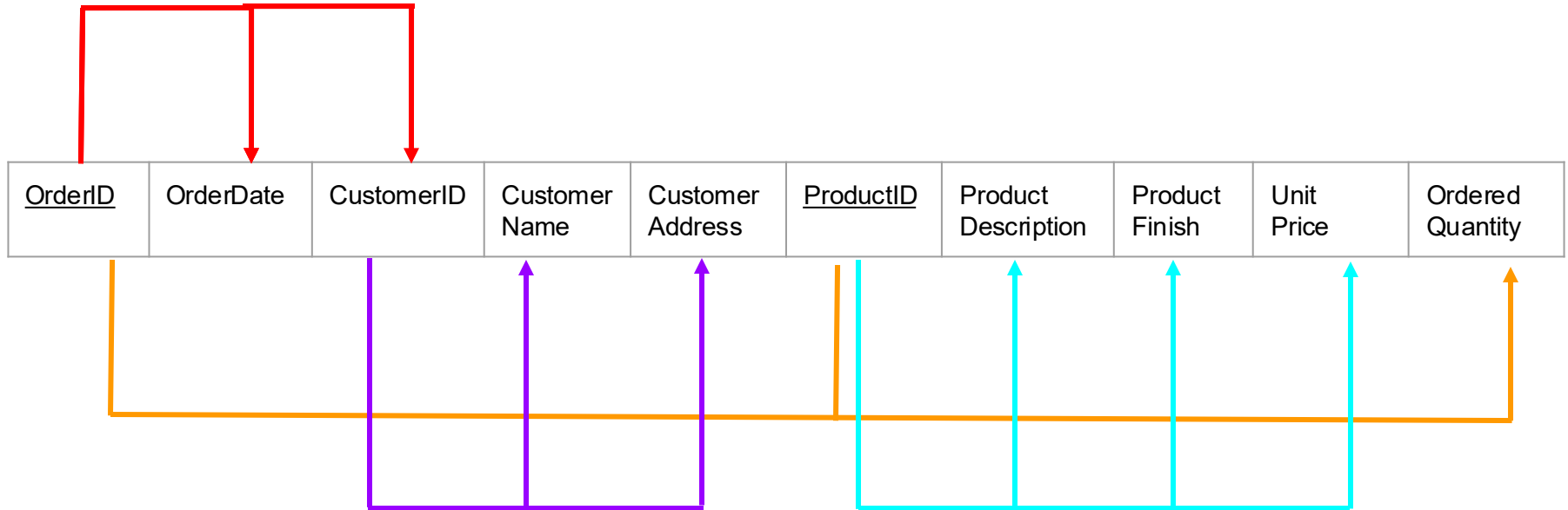
Formal Definition: A functional dependency $X \twoheadrightarrow Y$ is a **partial dependency** if some attribute $A \in X$ can be removed from X and the dependency still holds; that is, for some $A \in X$, $(X - \{A\}) \twoheadrightarrow Y$.

1. $\{\text{OrderID}, \text{ProductID}\} \rightarrow \{\text{OrderDate}\}$
2. $\{\text{OrderID}, \text{ProductID}\} \rightarrow \{\text{Unit_Price}\}$



Partial Functional Dependency

Proper handling of PFDs helps to eliminate redundancies, maintain data integrity (reduce data anomalies from insertion/deletion/update), and simplify the database structure as well as improve overall query performance.



Transitive Functional Dependency

A Transitive Functional Dependency is a scenario where a set of attribute (**Y**), is functionally dependent on another set of attributes (**Z**), which in turn is functionally dependent on the primary/unique key (**X**).

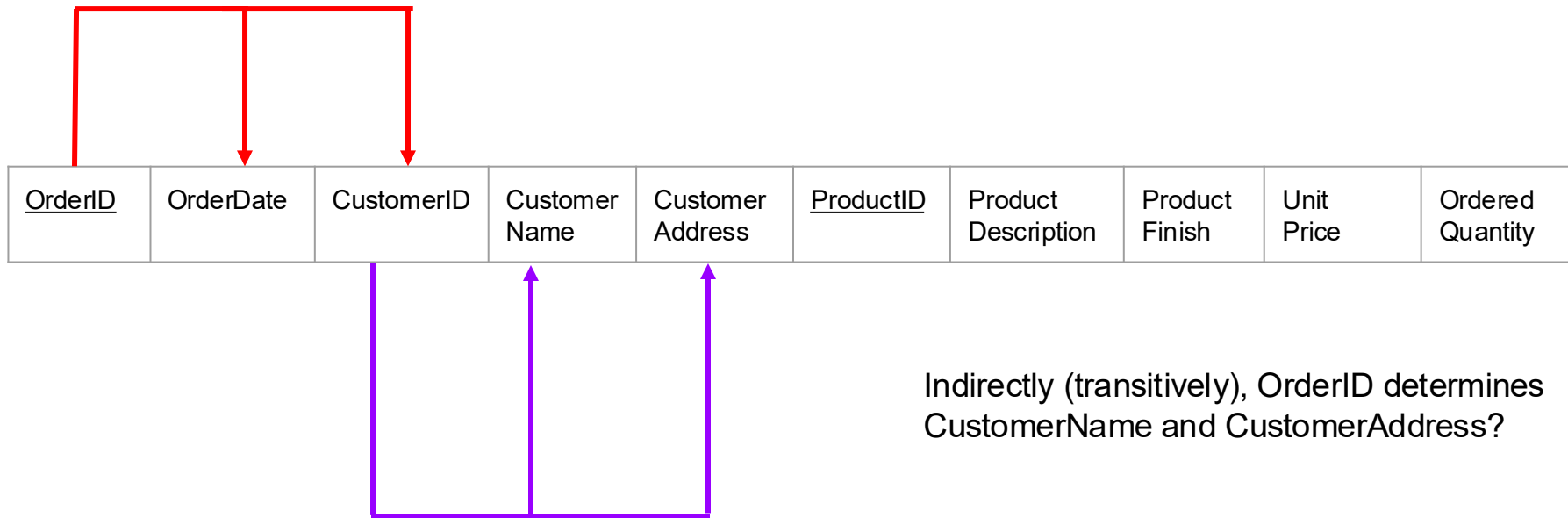
⇒ **$X \rightarrow Z$ and $Z \rightarrow Y$**

⇒ Usually, both **Z** and **Y** contain no prime attributes.

Such dependencies can be a sign that the data table should be further examined for potential redundancy and anomalies.

Transitive Functional Dependency

- $\text{OrderID} \rightarrow \{\text{CustomerID}\}$
- $\text{CustomerID} \rightarrow \{\text{CustomerName, CustomerAddress}\}$



A step-by-step process

Normalization

Process of analyzing and the given relations of a schema base of Functional Dependencies (*minimal cover preferred*) and Primary Keys to achieve the desirable properties of:

1. Minimizing redundancy
2. Minimizing insertion, deletion, and modification anomalies.
3. Ensure that each resulting table represents a complete, proper subject or theme.
4. Ensure that the database design is stable and scalable.

Normalization = dividing large tables into smaller, more manageable tables
(decomposition)

Denormalization

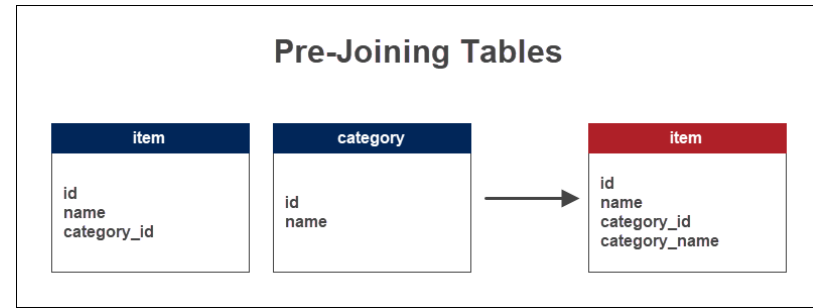
The process of storing the join of normalized relations as a base relation, which is in a lower normal form.

Deliberately reducing the “normalization level” of a database schema.

Intentionally introducing redundancy, why?

Could denormalization lead to faster query performance?

Denormalization is used for reporting purposes.



Review Keys

A Superkey is a subset of attributes with the property that no two tuples in any relation state r of the relation R has the same combination of values.

A Key is a Superkey such that removing attribute A from K makes K not a Superkey.

A Candidate Key is each key of a relation that has more than 1 key.

A Primary Key is the arbitrarily designated candidate key used to identify tuples.

A Unique Key is a Candidate Key that is not a Primary Key.

Understanding primary keys and candidate/unique keys is **essential** to the normalization process.

First Normal Form (1NF)

- Goal:
 - Eliminate repeating groups in tables
- Rules:
 - A table should have a proper primary key defined.
 - Each column must contain atomic (indivisible) values. No multi-valued attributes or sets are allowed.
 - Nested relations within a table are not allowed either.
 - All entries in a column must be of the same kind.

(a)

DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn	Dlocations



Not in 1NF because Dlocations is not an atomic attribute.

(b)

DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn	Dlocations
Research	5	333445555	{Bellaire, Sugarland, Houston}
Administration	4	987654321	{Stafford}
Headquarters	1	888665555	{Houston}

Is this in 1NF if the RDBMS supports list/vector data types?

(c)

DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn	<u>Dlocation</u>
Research	5	333445555	Bellaire
Research	5	333445555	Sugarland
Research	5	333445555	Houston
Administration	4	987654321	Stafford
Headquarters	1	888665555	Houston

One solution is to expand the key so that there will be a separate tuple for each location of a department.

Is this the **best** solution for normalizing a table into 1NF?

(a)

EMP_PROJ		Projs	
Ssn	Ename	Pnumber	Hours

Nested relation is simply a more complex form of a set.

(b)

EMP_PROJ

Ssn	Ename	Pnumber	Hours
123456789	Smith, John B.	1	32.5
		2	7.5
666884444	Narayan, Ramesh K.	3	40.0
453453453	English, Joyce A.	1	20.0
		2	20.0
333445555	Wong, Franklin T.	2	10.0
		3	10.0
		10	10.0
		20	10.0
999887777	Zelaya, Alicia J.	30	30.0
		10	10.0
987987987	Jabbar, Ahmad V.	10	35.0
		30	5.0
987654321	Wallace, Jennifer S.	30	20.0
		20	15.0
888665555	Borg, James E.	20	NULL

(c)

EMP_PROJ1

<u>Ssn</u>	Ename
------------	-------

EMP_PROJ2

<u>Ssn</u>	<u>Pnumber</u>	Hours
------------	----------------	-------

- Try to detect if a tuple has a relation-like structure within it.
- Example) EMP_PROJ(Ssn, Ename, **Pnumber**, **Hours**)
- Identify the nested relation: EMP_PROJ(Ssn, Ename, **{PROJS(Pnumber, Hours)}**)
- Not every DBMS disallows 1NF. Some support complex datatypes or complex objects to be nested/combined into a relation, i.e., *NoSQL databases*.
- Data such as images, documents, video clips, etc. is usually treated as an atomic value.
- **BLOB** (SQL) – binary large object (e.g., you don't manipulate individual pixels of an image)
- **CLOB** (SQL) – character large object (e.g., a document is treated as a singular item)

Second Normal Form (2NF)

Goal: Ensure that all non-key attributes are *fully dependent* on the **primary key**.

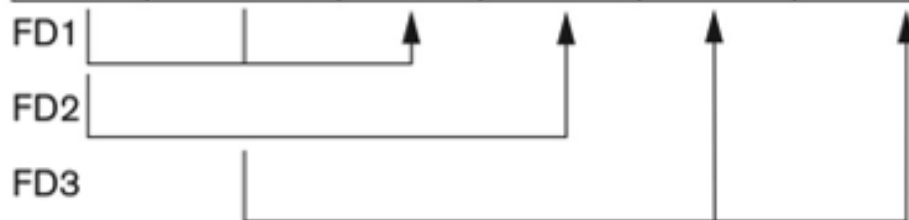
Rules:

- A Relation Schema R is in 2NF if it satisfies 1NF, and every non-prime Attribute A in R is Fully Functionally Dependent on the Primary Key of R.
- In other words, absolutely no partial dependencies of any column on the primary key.

(a)

EMP_PROJ

<u>Ssn</u>	<u>Pnumber</u>	Hours	Ename	Pname	Plocation
------------	----------------	-------	-------	-------	-----------



2NF Normalization

EP1

<u>Ssn</u>	<u>Pnumber</u>	Hours
------------	----------------	-------



EP2

<u>Ssn</u>	Ename
------------	-------



EP3

<u>Pnumber</u>	Pname	Plocation
----------------	-------	-----------



General Definition for 2NF

- What about Candidate Keys?
- A relation R is in 2NF if:
 - It is in 1NF.
 - No non-prime attribute is partially dependent on any candidate key.
- What if the relation does not have any composite candidate keys (i.e., all keys (primary + unique) consist of a single attribute)?

General Definition for 2NF

- What if the relation does not have any composite candidate keys (i.e., all keys (primary + unique) consist of a single attribute)?

<u>EmployeeID</u>	Name	Department	Salary
001	Alice	HR	50000
002	Bob	Marketing	55000

- **Given FDs:**
 - {EmployeeID} → {Name, Department, Salary}

General Definition for 2NF

<u>EmployeeID</u>	Name	Department	Salary
001	Alice	HR	50000
002	Bob	Marketing	55000

- **Given FDs:**

- $\{\text{EmployeeID}\} \rightarrow \{\text{Name}, \text{Department}, \text{Salary}\}$

PFDs cannot occur if a relation's primary/candidate keys contain only a single attribute. => automatically in 2NF!

General Definition for 2NF

Ideally, when testing for 2NF, it's important to consider ALL candidate keys, not just the primary key.

Why?

General Definition for 2NF

R(A, B, C, D, E, F, G) – Simple Example

Candidate Keys: {A, B}, {B, C, D}

FDs:

1. {A, B} $\rightarrow$ {C, D, E, F, G}
2. {B, C, D} $\rightarrow$ {A, E, F, G}
3. {B} $\rightarrow$ {E}
4. {G} $\rightarrow$ {F}

Is R in 2NF?

General Definition for 2NF

$R(A, B, C, D, E, F, G)$

Candidate Keys: $\{A, B\}$, $\{B, C, D\}$

FDs:

1. $\{A, B\} \rightarrow \{C, D, E, F, G\}$
2. $\{B, C, D\} \rightarrow \{A, E, F, G\}$
3. $\{B\} \rightarrow \{E\}$
4. $\{G\} \rightarrow \{F\}$

If not, show how to normalize R into 2NF.

General Definition for 2NF

R(A, B, C, D, E, F, G) – Challenging Example

Candidate Keys: {A, B}, {B, C, D}

FDs:

1. {A, B} $\rightarrow$ {C, D, E, F, G}
2. {B, C, D} $\rightarrow$ {A, E, F, G}
3. {B} $\rightarrow$ {E}
4. {G} $\rightarrow$ {E, F}

Is R in 2NF?

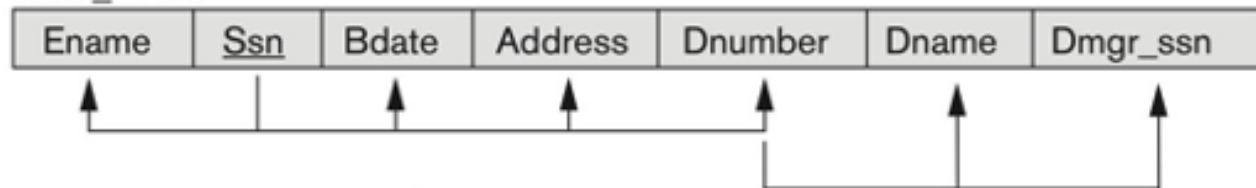
Third Normal Form (3NF)

A relationship schema is in 3NF if satisfies 2NF, and no non-prime attribute of R is **Transitively Dependent** on the primary key.

Recall that a transitive dependency is when a non-prime attribute depends on another non-prime attribute, which is indirectly dependent on the primary key.

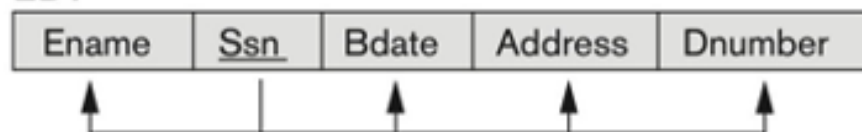
(b)

EMP_DEPT

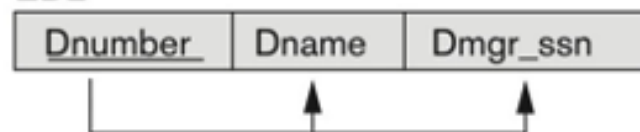


3NF Normalization

ED1



ED2



General Definition for 3NF

- A relation R is in 3NF if:
 - It is in 2NF
 - For every functional dependency $X \rightarrow Y$:
 - X is a superkey or,
 - Y is a prime attribute.

General Definition for 3NF

$R(A, B, C, D, E, F, G)$

Candidate Keys: $\{A, B\}$, $\{B, C, D\}$

FDs:

1. $\{A, B\} \rightarrow \{C, D, E, F, G\}$
2. $\{B, C, D\} \rightarrow \{A, E, F, G\}$
3. $\{F\} \rightarrow \{G\}$

Is R in 3NF? If not, normalize it.

General Definition for 3NF

R(A, B, C, D, E, F, G)

Candidate Keys: {A, B}, {B, C, D}

FDs:

1. $\{A, B\} \rightarrow \{C, D, E, F, G\}$
2. $\{B, C, D\} \rightarrow \{A, E, F, G\}$
3. $\{F\} \rightarrow \{C\}$
4. $\{G\} \rightarrow \{D\}$

Is R in 3NF? If not, normalize it.

BCNF

- Boyce-Codd Normal Form
 - Named after Raymond Royce and Edgar F. Codd
- A refinement of 3NF by catching certain anomalies left by 3NF and 2NF
- A relation R is in BCNF if, for every **non-trivial** functional dependency $X \rightarrow Y$ that holds over R:
 - X is a superkey.
 - That's it??

Example) 3NF anomalies

Candidate Keys:

1. {StudentID, Course}
2. {StudentID, Instructor}

Assumptions:

- A student can enroll in more than one subject.
 - A subject can be taught by multiple instructors.
 - Each instructor teaches only one subject.
-
- FD1: {StudentID, Course} $\rightarrow$ Instructor
 - FD2: {StudentID, Instructor} $\rightarrow$ Course
 - FD3: Instructor $\rightarrow$ Course

StudentID	Course	Instructor
101	Math	Mr. Smith
101	C++	Mr. Wyatt
102	Math	Mrs. Jones

Example) 3NF anomalies

Determine whether or not the relation is in 2NF.

- FD1: {StudentID, Course} → Instructor
- FD2: {StudentID, Instructor} → Course
- FD2: Instructor → Course

Candidate Keys:

1. {StudentID, Course}
2. {StudentID, Instructor}

StudentID	Course	Instructor
101	Math	Mr. Smith
101	C++	Mr. Wyatt
102	Math	Mrs. Jones
103	C++	Mr. Wyatt

Example) 3NF anomalies

Determine whether or not the relation is in 3NF.

- FD1: {StudentID, Course} → Instructor
- FD2: {StudentID, Instructor} → Course
- FD3: Instructor → Course

Candidate Keys:

1. {StudentID, Course}
2. {StudentID, Instructor}

StudentID	Course	Instructor
101	Math	Mr. Smith
101	C++	Mr. Wyatt
102	Math	Mrs. Jones
103	C++	Mr. Wyatt

Example) 3NF anomalies

Update Anomaly: If the instructor for a specific course changes, how many rows do we have to update?

Insertion Anomaly: Redundancy when inserting all the students enrolled in a new course?

Deletion Anomaly: If a student drops a course, and they are the last student in that course? Do we want to lose the course info?

StudentID	Course	Instructor
101	Math	Mr. Smith
101	C++	Mr. Wyatt
102	Math	Mrs. Jones
103	C++	Mr. Wyatt

Example) 3NF anomalies

Determine whether or not the relation is in BCNF.

- FD1: {StudentID, Course} → Instructor
- FD3: {StudentID, Instructor} → Course
- FD2: Instructor → Course

Candidate Keys:

1. {StudentID, Course}
2. {StudentID, Instructor}

StudentID	Course	Instructor
101	Math	Mr. Smith
101	C++	Mr. Wyatt
102	Math	Mrs. Jones
103	C++	Mr. Wyatt

Achieving BCNF

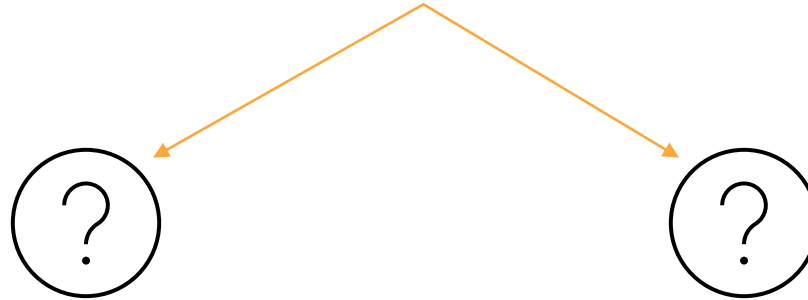
- Identify violations
- Decompose the table to eliminate these violations
- Repeat until all tables are in BCNF

General rule:

1. Let R be the relation not in BCNF, let $X \sqsubset R$, and let $X \rightarrow A$ be the FD that causes a violation of BCNF. R may be decomposed into two relations: $R - A$ and XA
2. If either $R - A$ or XA is not in BCNF, repeat the process.

Achieving BCNF

StudentID	Course	Instructor
101	Math	Mr. Smith
101	C++	Mr. Wyatt
102	Math	Mrs. Jones



Are all the FDs preserved? Is it okay?

The Nonadditive Join Property

NJB (Nonadditive Join Test for Binary Decompositions).

A decomposition $D = \{R_1, R_2\}$ of R has the lossless (nonadditive) join property with respect to a set of functional dependencies F on R *if and only if* either

- The FD $((R_1 \rightarrow R_2) \sqcap (R_1 \rightarrow R_2))$ exists in the original FD set
- The FD $((R_1 \rightarrow R_2) \sqcap (R_2 \rightarrow R_1))$ exists in the original FD set
- NJP enables information that may be **lost** (*of functional dependencies*) during decomposition to be **accurately** (nonaddictive) reconstructed.
- Maintain data and query integrity.
- Operational consistency: mitigate anomalies from insertion/update/delete operations.

BCNF

- In most cases, if a structured database design process is followed, most relations in 3NF are automatically in BCNF.
- If a relation is in BCNF, then is it automatically in 3NF? The converse?
- 3.5 Normal Form
- Decomposition might not be dependency-preserving.
- Prepares the relation for a higher level of normalization.

Multivalued Dependency and Fourth Normal Form

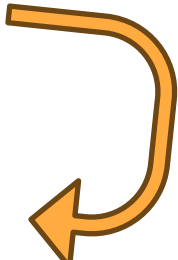
4NF is mainly concerned with removing undesirable redundancies caused by *multi-valued dependencies*. *Independent multi-valued attributes*

<u>Ename</u>	Pname	Dname
Smith	{X, Y}	{John, Anna}
Yosolo	{A, Y}	{Bob, Tim, Meri}

- Employee – Project (n:n)
- Employee – Dependent (n:n)
- Smith: 4 tuples containing redundant data
- Yosolo?

EMP

<u>Ename</u>	<u>Pname</u>	<u>Dname</u>
Smith	X	John
Smith	Y	Anna
Smith	X	Anna
Smith	Y	John

- 
- 1NF (# multivalued attribute > 1)
 - Multiple paths of normalization
 - Multivalued dependency might be introduced
 - Not often encountered in real-world applications

Join Dependencies and Fifth Normal Form

Example)

- SUPPLY is an all-key relation with *many-to-many relationships* between entities (Supplier, Parts, Projects)
- *Constraints:*
 - if a supplier **s** supplies part **p**
 - and if a project **j** uses part **p**
 - and if the supplier **s** supplies at least one part to project **j**
 - then supplier **s** will also be supplying part **p** to project **j**

SUPPLY

<u>Sname</u>	<u>Part_name</u>	<u>Proj_name</u>
Smith	Bolt	ProjX
Smith	Nut	ProjY
Adamsky	Bolt	ProjY
Walton	Nut	ProjZ
Adamsky	Nail	ProjX
Adamsky	Bolt	ProjX
Smith	Bolt	ProjY

Is there any redundancy?
Can SUPPLY be represented
by a set of smaller relations?

Join Dependencies and Fifth Normal Form

Join Dependency: A relation R has a join dependency if it's always true that R can be recreated by taking the natural join of its projections (subsets).

R_1

<u>Sname</u>	<u>Part_name</u>
Smith	Bolt
Smith	Nut
Adamsky	Bolt
Walton	Nut
Adamsky	Nail

R_2

<u>Sname</u>	<u>Proj_name</u>
Smith	ProjX
Smith	ProjY
Adamsky	ProjY
Walton	ProjZ
Adamsky	ProjX

R_3

<u>Part_name</u>	<u>Proj_name</u>
Bolt	ProjX
Nut	ProjY
Bolt	ProjY
Nut	ProjZ
Nail	ProjX

A join dependency is trivial if one of the projected subsets is equal to R because it does not specify any constraint on R.

Domain Key Normal Form (Simple Example)

Wealthy Person

<u>Wealthy Person</u>	Wealthy Person Type	Net Worth in Dollars
Steve	Millionaire	124,543,621
Roderick	Billionaire	6,553,228,893
Katrina	Billionaire	8,829,462,998
Gary	Millionaire	495,565,211

- **Primary Key:** {*Wealthy_Person*} ← **key constraint**
- **Domain Constraints:**
- **Wealthy_Person:** *Varchar*
- **Wealthy_Person_Type:** (*Millionaire, Billionaire*)
- **Net_Worth:** *Integer* >= 1,000,000
- **Any additional constraints?**

Suppose R is in
5NF, is it in
DKNF?

Domain Key Normal Form (Simple Example)

Wealthy Person

<u>Wealthy Person</u>	Net Worth in Dollars
Steve	124,543,621
Roderick	6,553,228,893
Katrina	8,829,462,998
Gary	495,565,211

Wealthiness Status

<u>Status</u>	Minimum	Maximum
Millionaire	1,000,000	999,999,999
Billionaire	1,000,000,000	999,999,999,999

DKNF Normalization: Remove the column `Wealthy\_Person\_Type` to eliminate the hidden constraint between the type and the net worth.

The original relation is still retrievable after joining the sub-relations based on the predicate.

Less prone to db operational anomalies.

Example) *What if a data entry mistake is made (Martin, Millionaire, 2,000,000,000)?*

Other Normal Forms

First Normal Form (1NF)

Second Form (2NF)

Third Normal Form (3NF)

Boyce-Codd Normal Forms (BCNF)

Fourth Normal Form (4NF)

Essential Tuple Normal Form (ETNF)

Fifth Normal Form (5NF)

Domain-key Normal Form (DKNF)

Sixth Normal Form (6NF)

Hierarchy of Normal Forms

	UNF (1970)	1NF (1970)	2NF (1971)	3NF (1971)	EKNF (1982)	BCNF (1974)	4NF (1977)	ETNF (2012)	5NF (1979)	DKNF (1981)	6NF (2003)
Primary key (no duplicate tuples)	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
No repeating groups	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Atomic columns (cells have single value)	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
No partial dependencies (values depend on the whole of every Candidate key)	✗	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓
No transitive dependencies (values depend only on Candidate keys)	✗	✗	✗	✓	✓	✓	✓	✓	✓	✓	✓
Every non-trivial functional dependency involves either a superkey or an elementary key's subkey	✗	✗	✗	✗	✓	✓	✓	✓	✓	✓	N/A
No redundancy from any functional dependency	✗	✗	✗	✗	✗	✓	✓	✓	✓	✓	N/A
Every non-trivial, multi-value dependency has a superkey	✗	✗	✗	✗	✗	✗	✓	✓	✓	✓	N/A
A component of every explicit join dependency is a superkey ^[8]	✗	✗	✗	✗	✗	✗	✗	✓	✓	✓	N/A
Every non-trivial join dependency is implied by a candidate key	✗	✗	✗	✗	✗	✗	✗	✗	✓	✓	N/A
Every constraint is a consequence of domain constraints and key constraints	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	N/A
Every join dependency is trivial	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓

References

Elmasri, R., & Navathe, S. B. (2017). Fundamentals of database systems, 7th ed. Harlow: Pearson Education.

Hoffer, J., Prescott, M., and McFadden, F. (2006). Modern Database Management (9th Edition). Prentice-Hall, Inc., Upper Saddle River, NJ, USA.

Ullman, J., Widom, J. (1997). A First Course in Database Systems. Prentice-Hall, Inc., Upper Saddle River, NJ, USA.